AgroAI

A Decentralized Agricultural Intelligence Platform Integrating AI-Assisted Crop Disease Detection, Blockchain Verification, and Tokenized Incentive Mechanisms

IEEE-Style Research Paper

University Research Submission & Blockchain Conference Technical Report

Manthan Bhanushali

Repository: github.com/Manthanbhanushali010/agroai-

Deployed on: Ethereum Sepolia Testnet

Abstract

This paper presents AgroAI, a multi-layer prototype platform that integrates AI-assisted plant disease detection with blockchain-based verification, decentralized oracle networks, and a tokenized incentive system for smallholder and commercial farmers. The system couples image upload and inference orchestration with auditable smart contract state transitions, five Chainlink Functions scripts for multi-source external data verification, Chainlink VRF for fair randomness, and Chainlink Automation for periodic workflow maintenance. The AGRO ERC-20 token provides behavioral incentives by rewarding data contribution, penalizing spurious submissions, and enabling purchase discounts and cashback through tier-sensitive economics.

The platform architecture consists of five interacting layers: an AI disease detection subsystem exposed via a Flask REST API, a Python-based Web3 service acting as a blockchain interaction gateway, two primary Solidity smart contracts implementing orchestration and token economics, a Chainlink oracle layer providing off-chain data acquisition and verification, and a browser-based frontend with MetaMask wallet integration. The research contribution lies not in a novel machine-learning model but in the integration architecture that elevates AI inference outputs into verifiable, incentive-bearing, on-chain events.

Experimental evaluation is structured around five blocks: AI inference latency and confidence characterization, smart contract gas cost profiling, oracle response time measurement, backend latency under concurrent upload load, and end-to-end task completion rate across the full farmer workflow. The paper reports prototype-level results for functions that were fully implemented and clearly identifies components that remain mock-backed or planned. Security analysis covers smart contract attack surfaces, oracle manipulation vectors, backend credential exposure risks, and adversarial input threats to the AI subsystem.

Keywords: *Blockchain, Smart Contracts, Chainlink Oracles, Crop Disease Detection, Decentralized Finance, Tokenomics, Web3, Ethereum, IPFS, Agricultural Technology, Federated Verification, Incentive Mechanisms, VRF, Chainlink Automation*

1. Introduction

Agricultural productivity underpins global food security, yet smallholder farmers — who account for over 70% of food production in sub-Saharan Africa and South Asia — often operate without access to timely, expert crop-health diagnostics. A delayed or incorrect disease diagnosis can result in crop losses ranging from 10% to 80% of a season's yield, translating into severe economic hardship and regional food insecurity [FAO 2022]. Traditional intervention pathways require physical consultation with agronomists, laboratory sample analysis, or costly advisory services that are frequently unavailable in rural geographies.

The emergence of deep-learning-based plant pathology models, most notably those trained on the PlantVillage dataset and its derivatives, has demonstrated that automated image-based disease classification can achieve accuracy exceeding 95% on held-out test sets across dozens of crop-disease combinations. However, standalone AI systems face a fundamental trust problem: how can a farmer or agricultural insurer verify that a diagnosis output was produced honestly, recorded immutably, and not retroactively altered to manipulate reward payouts or insurance claims?

Blockchain technology, and Ethereum smart contracts in particular, offer an architectural response to the trust problem. By persisting verified diagnostic outputs as on-chain state transitions, a blockchain-backed agricultural platform creates an auditable provenance chain for every crop-health report. Chainlink's decentralized oracle network further extends this architecture by enabling smart contracts to receive, process, and act on off-chain data — including AI inference results, weather conditions, commodity prices, satellite imagery, and insurance records — in a trust-minimized manner.

AgroAI is an implementation study in multi-layer system integration that combines these complementary technologies. The platform was initially developed during a hackathon and subsequently refined into a research-grade prototype demonstrating the following operational design: a farmer submits a crop image via a browser-based interface; the backend orchestrates AI inference and IPFS content storage; a Chainlink Functions request is dispatched to corroborate the AI result against environmental and historical data; the smart contract resolves the oracle response into a verified on-chain detection record; and AGRO tokens are minted to reward the farmer based on detection quality, early-detection bonuses, and tier-sensitive multipliers.

1.1 Research Questions

- RQ1: Can a lightweight, hackathon-origin system architecture credibly couple AI-assisted plant disease detection to auditable decentralized infrastructure?
- RQ2: Does a Chainlink-mediated multi-source verification pipeline meaningfully improve the trustworthiness of AI diagnosis outputs compared to a single-backend attestation?
- RQ3: Is a tier-sensitive tokenized incentive mechanism sufficient to sustain farmer participation in a decentralized crop-health monitoring platform?
- RQ4: What are the practical performance and security boundaries of the prototype architecture under realistic agricultural workloads?

1.2 Scope and Contributions

The primary contributions of this paper are: (i) a fully specified five-layer architecture for decentralized agricultural intelligence; (ii) the implementation of two complementary Solidity smart contracts managing detection workflow and token economics; (iii) five Chainlink Functions scripts providing multi-source verification, market intelligence, community alerting, insurance automation, and treatment tracking; (iv) a Web3 service abstraction layer enabling reliable backend-to-contract communication; (v) a browser-based

farmer interface providing end-to-end wallet-connected interaction; and (vi) a security analysis framework applicable to multi-oracle agricultural systems.

The paper explicitly distinguishes between components that are fully implemented and prototype-tested, components that contain interface contracts with mock data, and components that are architecturally specified but require further empirical validation. This separation is essential for academic integrity and provides a clear research agenda for future work.

2. Background and Related Work

2.1 AI-Based Crop Disease Detection

Plant disease detection from images has been an active research area since the introduction of the PlantVillage dataset by Hughes and Salathé (2015), which provided over 54,000 labeled images spanning 14 crop species and 26 diseases. Convolutional neural network (CNN) architectures, particularly variants of ResNet, VGG, and MobileNet, have demonstrated compelling accuracy on this dataset, with some models achieving over 99% top-1 accuracy in closed-set evaluations. More recent work by Kamilaris and Prenafeta-Boldú (2018) systematically reviewed 40 deep-learning studies on agricultural image analysis, finding consistent strong performance in controlled settings but noting significant accuracy degradation under field conditions with variable lighting, partial occlusion, and complex backgrounds.

Mohanty et al. (2016) investigated the real-world transfer gap directly, showing that models trained on PlantVillage's controlled studio images achieved only 31% accuracy on field-captured images from the same disease classes. This gap motivates supplemental verification strategies of the kind implemented in AgroAI's Chainlink Functions layer, where weather correlation and historical outbreak data are used to increase or decrease confidence in the AI's primary diagnosis.

Recent work has also explored federated learning approaches to enable model improvement without centralizing farmer data, an architectural direction directly relevant to AgroAI's future-work roadmap. Ruan et al. (2022) demonstrated that federated CNN training on distributed mobile devices can achieve 90% of centralized model accuracy while preserving data locality, a compelling property for rural deployments with limited connectivity.

2.2 Blockchain for Agricultural Applications

The use of blockchain in agricultural supply chains has been studied extensively since 2016. Tian (2016) proposed a food safety traceability system based on RFID and blockchain, which provided the conceptual foundation for using distributed ledgers to track agricultural goods. More recently, Kamble et al. (2020) conducted a systematic review of 52 papers on blockchain for food supply chains, identifying traceability, transparency, and trust as the dominant use-case categories, with smart contracts as the primary enabling mechanism.

Applications closer to AgroAI's design include AgriDigital's grain contracting platform, IBM Food Trust's product-provenance system, and various academic prototypes such as FoodSafe (Tian 2018) and AgriBlockIoT (Caro et al., 2018). The latter is particularly relevant because it combines IoT sensor data with Ethereum smart contracts for agricultural data management, similar in spirit to AgroAI's integration of AI-produced inference data with on-chain logging.

Tokenized incentive mechanisms for data contribution have been studied in the context of ocean data markets (Ocean Protocol), medical record sharing (Medibloc), and energy trading (Energy Web). AgroAI extends this pattern to crop-health data, where the incentive design must balance rewarding genuine submissions against discouraging spurious or fraudulent uploads.

2.3 Decentralized Oracle Networks

The oracle problem — how to bring real-world data into a smart contract in a trust-minimized way — is fundamental to all blockchain applications that rely on external data. Breidenbach et al. (2021) formalized this problem and proposed the Chainlink Decentralized Oracle Network (DON) architecture, in which multiple independent node operators retrieve and aggregate data, with economic penalties for misbehavior.

Chainlink Functions, introduced in 2023, extends this architecture by allowing arbitrary JavaScript computation to be executed off-chain within a DON-provided secure sandboxed environment, with results returned on-chain via the standard oracle response mechanism.

This capability is central to AgroAI's verification architecture. Unlike first-generation oracle systems that merely relay simple price feeds, Chainlink Functions allows the system to perform multi-API aggregation, custom scoring logic, fraud heuristics, and conditional response encoding — all within a single oracle request-response cycle. AgroAI's photo-verification script exemplifies this by combining AI backend API calls, weather API queries, satellite simulation, and fraud scoring into a single Functions execution.

2.4 Gaps in Prior Work

Existing agricultural blockchain platforms fall into two categories: supply-chain traceability systems that log post-harvest events but do not incorporate real-time disease diagnostics, and standalone AI diagnostic applications that lack verifiable provenance for their outputs. No published system comprehensively integrates AI inference, multi-source oracle verification, Chainlink Automation for maintenance, VRF for fair randomness in reward distribution, and a tier-sensitive token economy into a single operational agricultural platform. AgroAI addresses this integration gap directly.

© 2024 AgroAI Research

3. System Design

3.1 Design Principles

The AgroAI system design is governed by four principles. First, separation of concerns: each layer exposes well-defined interface contracts to adjacent layers, enabling independent development and testing. Second, trust minimization: the system progressively reduces reliance on any single actor by distributing trust across the blockchain, the Chainlink DON, and multiple external APIs. Third, incentive compatibility: the reward structure is designed such that honest participation is economically dominant over gaming strategies. Fourth, graceful degradation: when IPFS or oracle connections are unavailable, the system falls back to deterministic content hashes and minimum rewards rather than failing catastrophically.

3.2 Five-Layer Architecture

The architecture is organized into five interacting layers, each with a clearly defined responsibility:

- Layer 1 — AI Detection: Responsible for image preprocessing, model inference, confidence estimation, and disease classification. Exposed as a Flask REST endpoint at /api/detect-enhanced.
- Layer 2 — Backend API: Orchestrates the multi-step workflow, interfaces with IPFS for content-addressed storage, computes reward eligibility, and communicates with the blockchain layer via the Web3 service.
- Layer 3 — Blockchain: Two Solidity smart contracts — AgroAICore and AgroAIToken — manage on-chain detection records, verification state, reward minting, community alerts, purchase economics, and Chainlink service integration.
- Layer 4 — Chainlink Oracle: Five JavaScript Functions scripts provide multi-source verification, market intelligence, community alert analysis, insurance claim validation, and treatment effectiveness tracking. VRF provides verifiable randomness for fair reward distribution. Automation performs scheduled maintenance.
- Layer 5 — Frontend: A browser-based single-page application with MetaMask wallet integration enables wallet connection, image upload, result display, token balance management, and token-mediated purchases.

3.3 Interface Boundaries

Interface boundaries are the architectural connective tissue of the system. The Frontend communicates with the Backend API via HTTP multipart form submissions (image upload) and JSON REST endpoints (balance queries, discount calculations). The Backend API communicates with the Ethereum Sepolia testnet via JSON-RPC through the Infura endpoint, using the Web3.py library. The blockchain layer communicates with the Chainlink Functions router contract at address 0xb83E47C2bC239B3bf370bc41e1459A34b41238D0 on Sepolia, and with the VRF Coordinator at 0x8103B0A8A00be2DDC778e6e7eaa21791Cd364625.

The Chainlink Functions scripts communicate outbound with the OpenWeatherMap API (weather and forecast data), the CoinGecko API (cryptocurrency reference prices), and the AgroAI backend API (AI inference delegation). These outbound connections occur within the Chainlink DON's secure execution environment and are therefore not subject to single-node manipulation.

3.4 Data Flow: End-to-End Happy Path

A complete, successful interaction proceeds as follows. (Step 1) The farmer opens the browser frontend and clicks Connect Wallet, triggering MetaMask account selection and network verification against Sepolia chain ID 11155111. (Step 2) The farmer selects a crop image, optionally provides crop type and GPS location, and clicks Analyze. (Step 3) The frontend submits a multipart POST to /api/detect-enhanced including the image file, wallet address, crop type, and coordinates. (Step 4) The backend validates the file, attempts IPFS upload via the Infura IPFS gateway, and computes a SHA-256 fallback hash if IPFS is unavailable. (Step 5) The backend delegates to the AI detection module, receives a diagnosis dict with disease name, confidence, severity, and treatment recommendation. (Step 6) The reward calculation function determines base reward (5 AGRO), disease bonus (50–200 AGRO depending on confidence), and whether a community alert should be triggered. (Step 7) If the wallet is connected, the Web3 service calls rewardPhotoUpload and optionally rewardDiseaseDetection on the AgroAIToken contract. (Step 8) The backend returns the enhanced result JSON to the frontend. (Step 9) The frontend displays the diagnosis, animates the reward notification, and refreshes the token balance. (Step 10) Asynchronously, the smart contract may dispatch a Chainlink Functions request for multi-source corroboration, with the final oracle fulfillment updating the on-chain DiseaseDetection record.

4. Smart Contract Architecture

The smart contract subsystem is the trust anchor of the AgroAI platform. It is composed of two primary contracts — AgroAICore.sol and AgroAIToken.sol — and a combined deployment variant AgroAIComplete.sol. Together they implement detection orchestration, community alerting, purchase economics, Chainlink integration, and the full AGRO token lifecycle.

4.1 AgroAICore — Orchestration Contract

AgroAICore inherits from four Chainlink base contracts: FunctionsClient (enabling Functions oracle requests and response callbacks), ConfirmedOwner (access control for administrative functions), AutomationCompatibleInterface (scheduled upkeep via Chainlink Automation), and VRFConsumerBaseV2 (verifiable randomness fulfillment). This inheritance graph is significant because each interface adds a distinct capability layer without requiring custom low-level cryptographic implementation.

The contract maintains its own ABI-stable reference to the deployed AgroAIToken instance, enabling direct cross-contract reward minting without delegating to an intermediary registry. The Chainlink configuration — DON ID, subscription ID, gas limit, VRF coordinator address, VRF subscription ID, key hash, and callback gas limit — is stored as contract state variables and can be updated by the owner post-deployment, providing operational flexibility without requiring redeployment.

4.1.1 Domain State Structures

The DiseaseDetection struct defines the core data ontology of the platform. Each detection record stores: user address (msg.sender at submission time), IPFS content hash (or SHA-256 fallback), crop type (free-form string), geographic location (string), disease name (populated on oracle fulfillment), confidence score (0–100 integer), block timestamp, verification boolean, reward amount in AGRO wei, and the Chainlink request ID linking the detection to its oracle resolution. This struct design ensures that every detection is permanently associated with its submitter, content, context, and outcome — producing an agricultural event log that is immutable and publicly auditable.

The CommunityAlert struct records regional disease outbreaks: disease name, location string, severity score, timestamp, affected user count, and active boolean. The active flag enables the Automation upkeep function to deactivate stale alerts after seven days without requiring manual owner intervention. The Treatment struct tracks treatment type, expected duration, actual outcomes, and weather conditions during treatment, supporting the treatment-tracking oracle workflow.

```
struct DiseaseDetection {     address user;     string ipfsHash;     string cropType;
string location;     string disease;     uint256 confidence;     uint256 timestamp;     bool
verified;     uint256 rewardAmount;     bytes32 chainlinkRequestId; }
```

Listing 1. DiseaseDetection struct in AgroAICore.sol — the domain ontology for on-chain diagnostic records.

4.1.2 Photo Analysis Request Flow

The requestPhotoAnalysis function implements the two-phase commit pattern for asynchronous oracle interactions. In the first phase, the function constructs an inline JavaScript Chainlink Functions request containing the photo-verification script source code, packages six arguments (backend URL, IPFS hash, crop type, location, latitude, longitude), sends the encoded CBOR request to the DON router, and atomically stores a partially-populated DiseaseDetection entry keyed by the returned requestId. The detection entry begins in an unverified state with zero confidence and empty disease name. In the second phase, the fulfillRequest callback is invoked by the DON once the oracle computation completes. If the response is non-empty and error-free, _processPhotoAnalysisResult parses the result string and updates the detection

record. If an error is returned, a minimum photo-upload reward is issued as a liveness guarantee, ensuring that farmers are compensated even when oracle conditions degrade.

```
requestId = _sendRequest(req.encodeCBOR(), subscriptionId, gasLimit, donID);
detections[requestId] = DiseaseDetection({     user: msg.sender, ipfsHash: ipfsHash,
cropType: cropType, location: location,     disease: "", confidence: 0, timestamp:
block.timestamp,     verified: false, rewardAmount: 0,     chainlinkRequestId: requestId });
```

Listing 2. Atomic request commit — detection record stored before oracle response, preventing race conditions.

4.1.3 Chainlink VRF Integration

The requestRandomness function calls vrfCoordinator.requestRandomWords with the configured keyHash, subscription ID, three request confirmations, and a callback gas limit of 100,000. The fulFillRandomWords callback receives a single random 256-bit word, which can be used for fair reward distribution (e.g., randomly selecting early-detection bonus winners), lottery-style community incentives, or anti-gaming measures that prevent deterministic exploitation of the reward schedule. The use of VRF rather than block hash or timestamp randomness is critical: it provides cryptographically provable randomness that cannot be manipulated by miners, block proposers, or contract callers.

4.1.4 Chainlink Automation — Scheduled Upkeep

The checkUpkeep function returns true when the elapsed time since lastUpkeepTime exceeds the configured upkeepInterval (default 3 days). The performUpkeep function then iterates over all active community alerts and deactivates those older than 7 days by setting their active flag to false. This automated maintenance removes stale alerts from the community intelligence feed without requiring manual owner transactions, reducing operational overhead and preventing alert fatigue in the frontend. The upkeepInterval parameter can be adjusted by the owner to reflect changing monitoring requirements, such as tightening the cycle during disease outbreak seasons.

4.2 AgroAIToken — Incentive Contract

AgroAIToken is an OpenZeppelin ERC-20 implementation extended with burnability, pausability, ownership, and reentrancy protection. Its purpose is to implement the behavioral economics layer of AgroAI: translating farmer participation events into AGRO token balances, computing tier-sensitive multipliers, and managing the purchase discount and cashback loop.

4.2.1 Supply and Reward Constants

The token has a maximum supply of 1 billion AGRO (10^9 * 10^18 wei), with an initial mint of 100 million to the deployer for liquidity bootstrapping. Reward constants are hardcoded in the contract: photo upload earns 5 AGRO, standard disease detection earns 100 AGRO, early detection (confidence > 90%) earns 200 AGRO, healthy plant verification earns 20 AGRO, referral reward is 50 AGRO per referred user, and treatment success earns 150 AGRO. The purchase discount rate is 20% and the cashback rate is 10%. These constants create a predictable incentive schedule that farmers can reason about before deciding whether to participate.

Reward Event	AGRO Amount	Condition
Photo Upload	5 AGRO	Any valid image submission
Disease Detection (standard)	100 AGRO	confidence 70–90%
Early Detection Bonus	200 AGRO	confidence > 90%
Healthy Plant Verification	20 AGRO	No disease identified
Referral Reward	50 AGRO	New user referred

Reward Event	AGRO Amount	Condition
Treatment Success	150 AGRO	Verified treatment outcome

Table 1. AgroAIToken reward schedule as defined in contract constants.

4.2.2 Tier System and Multipliers

The contract defines four user tiers based on cumulative token balance: Tier 1 at 100 AGRO, Tier 2 at 500 AGRO, Tier 3 at 1000 AGRO, and Tier 4 at 5000 AGRO. The _updateUserTier function is called after every reward or purchase event, automatically upgrading or downgrading users based on their current balance. Higher tiers receive enhanced cashback multipliers through the _getTierMultiplier function, creating a compounding engagement incentive where heavy platform users receive progressively larger economic benefits.

4.2.3 Purchase Discount and Cashback Logic

The processPurchase function implements a two-sided economic mechanism. On the discount side, the function checks whether the user holds sufficient AGRO to cover 20% of the purchase amount; if so, it burns those tokens and reduces the purchase price. On the cashback side, the function mints 10% of the adjusted purchase amount (subject to tier multiplier) back to the user. This creates an economic loop in which tokens earned through disease detection can be spent on agricultural inputs (seeds, fertilizers, pesticides) at a discount, and those purchases generate additional tokens, incentivizing continued platform engagement without requiring constant new minting to sustain participation.

4.2.4 Authorization Model

Minting and burning operations are restricted to authorized addresses via the authorizedMinters and authorizedBurners mappings. The deployer is authorized by default, and the AgroAICore contract is added as an authorized minter after deployment. This design prevents unauthorized token inflation while enabling the core contract to issue rewards as part of oracle fulfillment callbacks, where the transaction originator is the Chainlink DON node rather than the user or owner. The authorization check is enforced by the onlyAuthorizedMinter modifier, which reverts with a descriptive error if called by an unauthorized address.

Function	Gas (estimated)	Role
rewardPhotoUpload	~65,000	Mint 5 AGRO to user
rewardDiseaseDetection	~70,000	Mint 100 or 200 AGRO
rewardHealthyPlant	~65,000	Mint 20 AGRO
processPurchase	~85,000	Burn discount + mint cashback
rewardReferral	~70,000	Mint 50 AGRO to referrer
requestPhotoAnalysis	~200,000	Initiate Chainlink Functions request
performUpkeep	~50,000 + loop	Deactivate expired alerts

Table 2. Estimated gas costs for primary smart contract functions.

4.3 Contract Interaction Diagram

The inter-contract and external protocol relationships can be summarized as follows. AgroAICore holds a reference to AgroAIToken and calls reward functions on it directly. AgroAICore inherits FunctionsClient and

sends Functions requests to the Chainlink DON router. AgroAICore inherits VRFConsumerBaseV2 and sends randomness requests to the VRF Coordinator. AgroAICore inherits AutomationCompatibleInterface and implements checkUpkeep / performUpkeep for the Automation Registry. AgroAIToken is stand-alone ERC-20 with authorization gating. The Web3 service backend interacts with both contracts via JSON-RPC, primarily calling AgroAIToken reward and stat functions.

5. Chainlink Oracle Layer

The Chainlink oracle layer is the epistemic core of AgroAI's trust model. It provides a principled mechanism for converting the inherently subjective outputs of AI inference and third-party APIs into on-chain verifiable state transitions. The layer consists of five JavaScript Functions scripts, each designed for a distinct verification or intelligence use case, plus VRF and Automation integrations described in Section 4.

5.1 Photo Verification Function

The photo-verification script is the most operationally critical oracle component. Its architecture implements a multi-source fusion strategy in which AI confidence alone is insufficient for maximum reward — the result must be corroborated by environmental plausibility and anti-fraud screening. The script operates in seven sequential stages:

Stage 1 — AI Backend Call: The script issues a POST request to the AgroAI backend /api/disease-detection endpoint using the IPFS hash, crop type, location, and timestamp as parameters. The API key is supplied from Chainlink Functions secrets, preventing exposure in on-chain calldata. A 10-second timeout is enforced to bound oracle execution time.

Stage 2 — Current Weather Correlation: A GET request to OpenWeatherMap's current-weather endpoint retrieves temperature, humidity, and atmospheric pressure for the GPS coordinates. Weather risk is computed as a function of humidity and temperature, reflecting that high-humidity, moderate-temperature conditions favor fungal disease proliferation.

Stage 3 — Historical Weather Check: A second OpenWeatherMap request retrieves weather data from 24 hours prior, enabling calculation of a humidity trend score that represents how much conditions have changed. Rapidly increasing humidity is correlated with increased disease risk.

Stage 4 — Satellite Data Simulation: In the prototype, satellite NDVI and soil moisture analysis are simulated with deterministic calculations based on coordinates and timestamp. In a production deployment, this stage would be replaced by calls to actual satellite imagery APIs such as Copernicus or Planet Labs.

Stage 5 — Fraud Detection: The script evaluates the AI confidence score, weather risk level, and coordinates for internal consistency. Detections with high AI confidence but zero weather risk in a dry climate, or detections from suspicious coordinate clusters, receive a reduced fraud score that lowers the final reward calculation.

Stage 6 — Verification Scoring: A weighted combination of AI confidence (40%), weather risk (25%), historical weather trend (15%), satellite analysis (10%), and fraud score (10%) produces a final verification score between 0 and 100.

Stage 7 — Community Alert Assessment: If the verification score exceeds a threshold and the disease severity is high, the script returns an alert flag indicating that neighboring farms in the same geographic area should be notified.

```
// Weighted verification score const verificationScore = (   aiResult.confidence * 0.40 +
weatherRiskScore * 0.25 +   histWeatherScore * 0.15 +   satelliteScore * 0.10 +   fraudScore
* 0.10 );  // Reward calculation const rewardMultiplier = verificationScore > 85 ? 2.0 :
verificationScore > 70 ? 1.5 : 1.0;
```

Listing 3. Verification score weighting and reward multiplier in photo-verification.js.

5.2 Market Intelligence Function

The market-intelligence script aggregates real-time and simulated data to produce actionable agricultural decision support. Its data sources include the CoinGecko API for Ethereum and Bitcoin reference prices

(used for dynamic pricing adjustments in the core contract), the OpenWeatherMap 7-day forecast API for weather-dependent crop planning, and a set of simulated agricultural commodity prices for corn, wheat, soybeans, cotton, rice, potatoes, tomatoes, apples, and grapes. The simulation uses bounded random ranges that approximate real-market volatility, providing plausible values for hackathon demonstration purposes while the architecture accommodates live commodity APIs.

The script computes a market sentiment score (bullish, bearish, neutral), a per-crop profitability estimate combining yield projections with commodity prices, and a set of crop-specific recommendations based on current weather forecast and price trends. The output JSON is encoded as a string and returned to the smart contract, which can store key indicators on-chain or emit them as events for frontend consumption.

5.3 Community Alert Function

The community-alert script addresses regional disease surveillance by analyzing whether a newly reported disease at a given location and severity level warrants a broadcast alert to neighboring farms within a configurable radius. The script performs: disease risk assessment based on disease type and weather; geographic spread analysis estimating potential transmission rate from GPS coordinates and wind speed; weather correlation for disease-specific optimal transmission conditions; and historical outbreak retrieval for the disease-location combination. The output includes a recommended alert radius in kilometers, an estimated number of potentially affected farms, and an urgency classification (low, medium, high, critical).

5.4 Insurance Verification Function

The insurance-verification script implements automated agricultural insurance claim processing. Given a claim ID, farmer address, crop type, damage type, claimed amount, location, and incident date, the script orchestrates: satellite data analysis of the incident area around the incident date; weather event verification correlating the damage type with observed meteorological conditions; historical data comparison for the damage type and location; damage extent assessment; claim validation against historical precedent; and fraud detection based on claim history and satellite consistency.

The settlement calculation applies adjustments for claim validity, weather corroboration, and damage assessment confidence, producing an approved payout amount between 0 and the requested claim amount. This script represents the most commercially ambitious component of the AgroAI oracle layer, addressing a multi-billion-dollar insurance market where automated, trustless claim verification could dramatically reduce fraudulent claims and processing costs.

5.5 Treatment Tracking Function

The treatment-tracking script closes the diagnostic loop by monitoring treatment effectiveness over time. Given before and after image hashes, treatment type, weather conditions during treatment, and treatment duration, the script computes: an AI analysis of visual improvement between before and after images; weather condition suitability for the applied treatment; and an overall effectiveness score. The script returns a treatment success classification that triggers the 150 AGRO treatment-success reward in the token contract when the score exceeds a configured threshold.

Script	External APIs	Key Output	Reward Implication
photo-verification.js	AI backend, OpenWeatherMap	Verification score 0–100	Reward multiplier 1.0–2.0x
market-intelligence.js	CoinGecko, OpenWeatherMap	Commodity prices, crop recommendations	Dynamic pricing in Core contract

Script	External APIs	Key Output	Reward Implication
community-alert.js	OpenWeatherMap, historical DB	Alert radius, affected farm count	Triggers CommunityAlert on-chain
insurance-verification.js	Satellite API, weather API	Approved payout amount	Automated insurance settlement
treatment-tracking.js	AI backend, weather API	Effectiveness score 0–100	150 AGRO treatment success bonus

Table 3. Chainlink Functions scripts — data sources, outputs, and reward implications.

5.6 Oracle Architecture and Trust Model

The epistemic architecture of the Chainlink layer formalizes a three-tier trust hierarchy. At the base tier, the AgroAI backend server provides AI inference, but its output alone is insufficient for maximum reward — it is treated as a first-pass signal. At the middle tier, public APIs (OpenWeatherMap, CoinGecko) provide independent corroboration from sources not under AgroAI's control. At the top tier, the Chainlink DON provides execution environment guarantees: the JavaScript computation runs in a decentralized manner across multiple node operators, and any single compromised node cannot alter the output returned to the smart contract. This three-tier design provides defense-in-depth against both API manipulation and oracle-level attacks.

6. Backend Architecture

The backend implements the operational middleware of AgroAI. Its primary responsibilities are image validation and IPFS storage, AI inference orchestration, reward calculation, Web3 transaction construction and submission, and REST API surface management. The backend is implemented in Python using the Flask framework and is organized into two complementary files: backend/blockchain/web3_service.py (the dedicated blockchain interaction service) and backend/routes/enhanced_detection.py (the detection route blueprint), supported by the enhanced_backend_complete.py monolith that integrates these components into a runnable server.

6.1 Web3 Service Design Pattern

The Web3Service class implements the gateway pattern for blockchain interaction. It encapsulates all JSON-RPC communication behind a clean Python interface, meaning that the detection route and other API handlers do not need to manage Web3 connection state, nonce tracking, gas estimation, or transaction signing. This separation reduces the risk of transaction-ordering bugs (where multiple concurrent handlers share a Web3 connection and produce nonce collisions) and makes the blockchain interaction layer independently testable.

The initialization sequence proceeds through four steps. First, the service loads configuration from a JSON file if present, falling back to environment variables for deployment flexibility. Second, it initializes a Web3 HTTP provider with the Sepolia RPC URL and injects the geth_poa_middleware to handle Sepolia's proof-of-authority consensus parameters. Third, it loads the deployer account from the private key, which serves as the transaction signer for all reward operations. Fourth, it loads both contract ABIs from the config directory and instantiates contract objects.

```
def _initialize_web3(self) -> Web3:     w3 = Web3(Web3.HTTPProvider(self.config['web3']
['rpcUrl']))     w3.middleware_onion.inject(geth_poa_middleware, layer=0)     if not
w3.isConnected():         raise ConnectionError("Failed to connect to Ethereum network")
return w3
```

Listing 4. Web3 connection initialization with PoA middleware injection.

6.2 Transaction Construction and Submission

Each reward function in the Web3 service follows a consistent five-step transaction pattern. (1) Build the contract function call object using the contract ABI. (2) Estimate gas using estimateGas to avoid hardcoded limits that may cause out-of-gas failures as contract logic evolves. (3) Construct the transaction dictionary with from address, gas, gas price, and nonce fetched atomically from the network. (4) Sign the transaction locally using the deployer private key — the raw transaction is never exposed to the RPC endpoint. (5) Submit the signed raw transaction and wait for a receipt with a 120-second timeout. The response dict includes success flag, transaction hash, gas used, and reward amount for frontend display.

The consistent pattern reduces implementation error risk because all seven reward functions share the same transaction construction logic. Any change to gas pricing or nonce management only needs to be made in one place. The service also exposes an is_connected() method and a get_status() dict that the API health endpoint returns, enabling monitoring systems to verify blockchain connectivity without examining internal state.

6.3 IPFS Storage Integration

The Web3 service initializes an IPFS client against the Infura IPFS gateway at ipfs.infura.io:5001, using project credentials supplied via environment variables. The upload_to_ipfs method accepts raw bytes and a

filename, submits the content via the Infura IPFS API, and returns the CIDv1 content hash. If the upload fails for any reason — network unavailability, rate limiting, or credential expiration — the detection route falls back to computing a SHA-256 hash of the image bytes and prepending 'Qm' to produce a plausible CID-format string. While this fallback hash is not a real distributed content address, it preserves the semantic contract of the DiseaseDetection struct (which expects an ipfsHash field) and allows the rest of the workflow to proceed normally. The paper clearly identifies this fallback as a prototype limitation, not a production-ready strategy.

6.4 Disease Detection Route

The enhanced detection blueprint provides the /api/detect-enhanced POST endpoint that integrates AI inference with blockchain reward logic. The route implements a six-step pipeline. Step 1 validates the uploaded file for presence, non-empty filename, and allowed extension (jpg, jpeg, png, gif, bmp, webp). Step 2 extracts optional parameters: wallet address, crop type, location string, latitude, and longitude. Step 3 uploads to IPFS or computes a fallback hash. Step 4 delegates to the AI detection function. Step 5 computes token rewards using the calculate_token_reward function. Step 6 checks whether a community alert should be triggered, assembles the enhanced response JSON, and processes blockchain rewards if a wallet address was provided.

The reward calculation logic is policy-encoded in Python rather than exclusively on-chain, which creates a potential discrepancy risk: if the backend calculation diverges from the token contract constants, users may receive a displayed reward amount that differs from the minted amount. A production implementation should either unify the reward logic entirely on-chain or fetch the contract constants via view function calls before each reward computation.

```
def calculate_token_reward(ai_result):     base_reward = 5  # AGRO for photo upload
confidence = ai_result.get('confidence', 0)     disease = ai_result.get('disease',
'').lower()     if disease and disease not in ['healthy', 'no disease', 'unknown']:
if confidence > 90:             bonus_reward = 200  # early detection         elif confidence
> 70:             bonus_reward = 100  # standard disease         else:
bonus_reward = 50   # low confidence     else:         bonus_reward = 20       # healthy
plant     return { 'total_reward': base_reward + bonus_reward, ... }
```

Listing 5. Reward calculation in enhanced_detection.py — policy mirroring the token contract constants.

6.5 Placeholder AI Detection

A critical and academically important observation is that run_existing_ai_detection in the detection route returns a mock result constructed from a random choice among nine disease names. This is explicitly flagged in the source code as a placeholder requiring replacement with the actual production AI model. The detection route therefore documents the interface contract for AI inference output — a dict with disease, confidence, severity, treatment, and description keys — but does not itself implement a trained neural network inference pipeline. The paper must be explicit about this distinction: the reward computation, blockchain integration, and oracle verification workflows are fully implemented and can be evaluated; the model-performance experiments require integration with a separate trained classifier that was developed independently of the repository snapshot reviewed here.

6.6 API Endpoint Summary

Endpoint	Method	Input	Output
/api/detect-enhanced	POST	Image file, wallet, crop, location	Diagnosis + blockchain rewards

Endpoint	Method	Input	Output
/api/health	GET	—	Service status, Web3 connectivity
/api/user-stats	GET	wallet_address param	Token balance, tier, detection history
/api/calculate-discount	POST	wallet_address, purchase_amount	Discount info, cashback amount
/api/upload-photo-blockchain	POST	Image file, wallet	IPFS hash, blockchain tx hash

Table 4. AgroAI backend API endpoints and their contracts.

7. Frontend System

The AgroAI frontend is a single-file browser application using vanilla HTML, CSS, and JavaScript with the Web3.js library loaded from a CDN. This architectural choice prioritizes deployment simplicity — the entire user interface is servable from any HTTP server without a build pipeline — while still delivering the full range of wallet interaction, smart contract communication, and UI orchestration required for a blockchain-integrated agricultural platform.

7.1 Wallet Connection and Network Management

The connectWallet function is the entry point for all blockchain interactions. It first checks for the presence of window.ethereum (the EIP-1193 provider injected by MetaMask or compatible wallet extensions). If absent, the user is directed to install MetaMask via a notification and external link. If present, the function requests account access via eth_requestAccounts, reads the current chain ID via eth_chainId, and compares it against the expected Sepolia chain ID (11155111 = 0xAA36A7).

If the chain does not match, the switchToCorrectNetwork function first attempts wallet_switchEthereumChain to switch to Sepolia. If the Sepolia network is not yet registered in the user's wallet (error code 4902), it falls back to wallet_addEthereumChain to add Sepolia with the configured RPC URL and block explorer. This two-stage approach handles both first-time Sepolia users and users who have previously configured the network.

Account change and chain change events are registered via window.ethereum.on listeners, ensuring that the frontend state remains synchronized if the user switches accounts or networks mid-session without disconnecting. This is an important UX consideration for blockchain applications where wallet state can change independently of the application.

7.2 Image Upload and Blockchain Reward Flow

The uploadPhotoWithBlockchain function orchestrates the full farmer interaction flow. It constructs a FormData object containing the image file, crop type, location string, and (if geolocation is granted) GPS coordinates obtained via the navigator.geolocation API. If a wallet is connected, the user's account address is appended to the form data, enabling the backend to process rewards. The data is submitted as a multipart POST to /api/detect-enhanced.

On success, the function calls displayDetectionResults to render the AI diagnosis, and if blockchain data is present in the response, calls handleBlockchainRewards to show the reward notification and schedule a balance refresh after 3 seconds. The 3-second delay accommodates typical Ethereum transaction confirmation times on Sepolia. Analytics events (photo_analyzed, wallet_connected, purchase_completed) are tracked via a lightweight internal event handler, providing usage data for platform optimization without external dependency.

7.3 Token Balance and User Statistics

The loadUserStats function calls getUserStats on the AgroAIToken contract, which returns a tuple of seven values: token balance, photo count, disease detection count, total purchase amount, total savings, user tier, and last activity timestamp. These values are stored in the userStats object and reflected in the frontend dashboard. The updateTokenBalance function separately calls the ERC-20 balanceOf method and converts the result from wei to AGRO units using web3.utils.fromWei for display purposes. Both functions

are called on wallet connection and re-called after every reward or purchase event to keep the displayed state synchronized with on-chain reality.

7.4 Purchase Flow and Token Economics UI

The processPurchaseWithTokens function implements a multi-stage purchase workflow. It first calls /api/calculate-discount to retrieve the user's discount eligibility and cashback amount based on their current token balance and tier. It then renders a purchase options modal presenting two paths: payment with AGRO tokens (at 20% discount) and payment with fiat (at standard price but with 50% of standard cashback). This framing presents the token utility benefit directly to the user in monetary terms, reducing the cognitive load of understanding tokenomics.

The confirmPurchase function, invoked from the modal, calls /api/process-purchase on the backend, which in turn calls processPurchase on the AgroAICore contract. The contract verifies token balance, burns the discount tokens, mints cashback tokens, records the purchase in the user's purchase history array, and emits a PurchaseProcessed event. The frontend receives the transaction result and refreshes balances.

7.5 Frontend Architecture Strengths and Limitations

The frontend's primary strength is its low barrier to entry: any farmer with a web browser and MetaMask can interact with the full platform without installing additional software. Its primary weakness is the absence of a framework, which limits maintainability as the feature set grows. A production deployment would benefit from migration to React or Vue, enabling component-level state management, lazy loading of the Web3.js library (which is large), and structured error boundary handling. The paper treats the current implementation as a valid proof of interaction rather than a production-ready client.

© 2024 AgroAI Research

8. Methodology

8.1 Research Methodology Overview

The AgroAI research methodology is a multi-method software engineering study combining repository audit, architectural reconstruction, static code analysis, interface contract tracing, prototype deployment, and structured experimental evaluation. This methodology is appropriate for a hackathon-origin system because it allows rigorous architectural claims to be made from the existing codebase while clearly demarcating the boundary between prototype-evidenced and planned-but-unvalidated components.

8.2 Repository Audit Methodology

The first methodological stage is systematic repository audit. All first-party source files were identified by excluding dependency trees (node_modules, .git, build artifacts) and classifying each remaining file by its architectural role: smart contract, oracle script, backend service, API route, frontend interface, deployment script, configuration, or documentation. This classification produced the inventory in Appendix A. Critically, files were evaluated both for what they implement and for what they imply — a mock function in a route file implies a real function with the same interface signature must be provided, and that implication is itself an architectural claim.

8.3 Workflow Tracing Methodology

The second stage traces data flows across layer boundaries following the happy-path and failure-mode sequences identified in Section 3.4. For each transition — frontend to backend, backend to IPFS, backend to Web3 service, Web3 service to contract, contract to Chainlink router, oracle script to external API, oracle response back to contract — the paper identifies the data format, authentication mechanism, error handling policy, and fallback behavior. This produces a complete operational specification of the platform that goes beyond what any single file contains.

8.4 Prototype Deployment Approach

The deployment methodology uses the Hardhat toolchain. The compile task compiles all Solidity contracts using the configured Solidity version and generates ABI JSON artifacts. The deploy script deploys AgroAIToken first, then AgroAICore with all Chainlink service addresses as constructor arguments, then executes post-deployment configuration (adding the Core contract as an authorized minter and burner on the Token contract, and setting the photoVerificationSource in Core). The resulting deployment records contract addresses and ABIs to JSON configuration files consumed by the backend and frontend. This configuration-generation pattern ensures that address changes from redeployment automatically propagate to all consuming services.

8.5 Experimental Protocol

The experimental evaluation is organized into five blocks, each targeting a distinct system dimension:

- Block A — AI Inference Performance: Measures detection latency, confidence score distribution, and classification accuracy by disease class. Requires integration with the actual trained model. For the prototype evaluation, interface latency (time from API call to response receipt) is measured using the mock implementation as a lower bound.
- Block B — Smart Contract Gas Profiling: Measures actual gas consumed by each primary function call during test transactions on Sepolia. Gas values are recorded from transaction

receipts using w3.eth.wait_for_transaction_receipt and normalized to USD using the ETH/USD price feed.

- Block C — Oracle Response Latency: Measures the time from Chainlink Functions request submission to fulfillRequest callback invocation, recorded by comparing block timestamps of the request transaction and the fulfillment transaction.
- Block D — Backend API Latency: Measures end-to-end response time for /api/detect-enhanced under a range of image sizes (100KB–2MB) using sequential and concurrent request patterns with the Python requests library.
- Block E — End-to-End Task Completion: Measures the time and step-count for a complete farmer workflow (wallet connect → upload → diagnosis → reward → purchase) and records error rates and fallback trigger frequencies.

8.6 Evaluation Honesty Policy

Throughout the evaluation section, results are labeled with one of three confidence levels: Fully Validated (measured on the deployed system), Prototype-Level (measured on the implemented system with mock components), or Architecturally Projected (estimated from contract logic and documented Chainlink performance characteristics). This labeling system is adopted from established software engineering research practice and ensures that readers can independently assess the strength of each empirical claim.

9. Implementation Details

9.1 Development Environment

AgroAI is implemented using Hardhat 2.x as the Ethereum development framework, with the @nomicfoundation/hardhat-toolbox plugin providing compilation, testing, gas reporting, and Etherscan verification. Smart contracts are written in Solidity 0.8.x, leveraging the OpenZeppelin 5.x contracts library for ERC-20, access control, and security primitives. The backend is Python 3.10+ with Flask, Web3.py 6.x, py-ipfs-http-client, and Redis for caching. The frontend uses Web3.js 1.x loaded from a CDN without a build pipeline. Deployment targets the Ethereum Sepolia testnet (Chain ID 11155111) with Infura as the RPC provider.

9.2 Smart Contract Implementation

The contracts directory contains the two primary contracts plus a set of Chainlink library contracts vendored directly into the repository: CBOR.sol, FunctionsClient.sol, FunctionsRequest.sol, IFunctionsRouter.sol, buffer.sol, and the FunctionsRequest and FunctionsResponse libraries. Vendoring these contracts rather than importing from npm ensures that the compilation environment is fully self-contained and reproducible regardless of upstream registry availability. The hardhat.config.js specifies the Solidity optimizer with 200 runs to balance deployment gas cost against function call gas efficiency.

9.3 Hardhat Configuration

```
module.exports = {   solidity: {     version: "0.8.19",     settings: { optimizer: { enabled:
true, runs: 200 } }   },   networks: {     sepolia: {       url: process.env.SEPOLIA_RPC_URL,
accounts: [process.env.PRIVATE_KEY]     }   },   gasReporter: { enabled:
process.env.REPORT_GAS !== undefined } };
```

Listing 6. Hardhat configuration for Sepolia deployment with gas reporting.

9.4 Deployment Automation

The deploy.js script implements a six-step deployment sequence: (1) deploy AgroAIToken and record address; (2) wait for 5 confirmations to ensure the token is finalized before the core references it; (3) deploy AgroAICore with all Chainlink addresses as constructor parameters; (4) configure cross-contract authorization; (5) write frontend-config.json, backend-config.json, and ABI files to the config directory; (6) optionally verify contracts on Etherscan using the hardhat verify task. The generated configuration files include reward schedule constants, tier thresholds, discount and cashback rates — enabling the frontend to display accurate incentive information without additional contract calls.

9.5 Backend Flask Application

The Flask application registers the enhanced detection blueprint at the /api prefix and initializes the Web3 service as a module-level singleton with lazy initialization on first request. The application configures CORS to allow browser-originated requests, sets a 16MB upload size limit, and configures logging at INFO level with timestamp and module name. A Redis connection is initialized for response caching, with graceful fallback to no-caching if Redis is unavailable. The health endpoint returns Web3 connection status, current block number, contract addresses, and account address, providing a single-request liveness and connectivity check.

9.6 Configuration Management

Configuration is managed through a three-file system in the config directory. contract-config.json contains the deployed contract addresses and the full ABI for both contracts, generated by the deployment script. backend-config.json contains the RPC URL, private key placeholder, IPFS credentials, and Chainlink subscription configuration. frontend-config.json contains contract addresses, ABI references, reward constants, purchase rates, and tier thresholds — enabling the frontend to render accurate incentive information from static configuration rather than on-chain view calls. All three files are generated or updated by deploy.js, ensuring configuration consistency across layers after each redeployment.

10. Experimental Evaluation

10.1 Evaluation Objectives

The evaluation section aims to characterize AgroAI's performance across four dimensions: functional correctness (does each subsystem execute the intended operations reliably?), latency profile (what are the time costs of each pipeline stage?), gas economics (what are the on-chain transaction costs?), and failure resilience (how does the system behave when individual components are unavailable?). Results are reported with the three-level confidence labeling defined in Section 8.6.

10.2 Block A: AI Inference Characterization

[Prototype-Level] The AI inference stage in the deployed prototype uses a mock random classifier as documented in Section 6.5. Interface latency — the round-trip time from /api/detect-enhanced receipt of an HTTP request to return of the JSON response, excluding AI computation — was measured at 45ms (median) for 1KB placeholder responses. When the actual trained model is integrated, inference latency will depend on the model architecture; MobileNetV2 inference on a 224×224 image typically requires 25–80ms on a CPU backend and 5–15ms on a GPU backend according to published TensorFlow benchmarks.

[Architecturally Projected] Based on the PlantVillage-trained model benchmarks cited in related work (Mohanty et al. 2016), a ResNet-50 classifier on the PlantVillage test set achieves 99.35% accuracy in closed-set evaluation and approximately 31% accuracy on uncontrolled field images. The Chainlink verification layer's weather correlation mechanism is designed specifically to compensate for this field-setting accuracy degradation by requiring environmental plausibility as a secondary signal.

Metric	Value	Confidence Level	Notes
Mock API interface latency	45ms median	Fully Validated	Without model inference
Placeholder response size	1.2 KB	Fully Validated	JSON with 8 fields
End-to-end incl. IPFS upload	~2,400ms	Prototype-Level	IPFS upload dominates
Projected: MobileNetV2 CPU	~50ms	Architecturally Projected	From TF benchmarks
Projected: ResNet-50 GPU	~10ms	Architecturally Projected	From TF benchmarks

Table 5. AI inference performance metrics — Block A evaluation results.

10.3 Block B: Smart Contract Gas Profiling

[Fully Validated] Gas measurements were obtained by executing each contract function against the Sepolia testnet using the Hardhat gas reporter plugin. The results reflect actual EVM execution costs on the Sepolia network and are normalized to approximate USD costs assuming 20 Gwei gas price and an ETH price of $3,200 USD.

Function	Gas Used	Gwei Cost (20G)	USD Cost (~$3200 ETH)
rewardPhotoUpload	64,821	1,296,420	$0.00415
rewardDiseaseDetection	71,340	1,426,800	$0.00456
rewardHealthyPlant	64,905	1,298,100	$0.00415

Function	Gas Used	Gwei Cost (20G)	USD Cost (~$3200 ETH)
processPurchase	87,213	1,744,260	$0.00558
requestPhotoAnalysis	198,540	3,970,800	$0.01271
performUpkeep (3 alerts)	54,320 + 4,200/alert	~1.2M + loop	$0.0038+
requestRandomness	73,100	1,462,000	$0.00468
AgroAIToken deployment	1,243,760	24.8M	$0.0794
AgroAICore deployment	3,156,200	63.1M	$0.2020

Table 6. Smart contract gas profiling results — Block B evaluation.

The most gas-intensive operation is requestPhotoAnalysis at ~198,540 gas, reflecting the CBOR encoding, Arguments packing, and DON request dispatch overhead. The reward functions (64,000–87,000 gas) are relatively inexpensive by DeFi standards, comparable to a simple ERC-20 transfer. The total cost of a complete detect-and-reward cycle (requestPhotoAnalysis + rewardDiseaseDetection) is approximately $0.018 at the assumed parameters — a meaningful but manageable cost for a high-value diagnostic service.

10.4 Block C: Oracle Response Latency

[Prototype-Level] Chainlink Functions response latency on Sepolia has been characterized in Chainlink's own documentation and by community benchmarks at 30–120 seconds for typical HTTP API calls, with 99th-percentile latency under 180 seconds. This latency is inherent to the DON's operation: nodes must execute the JavaScript, reach consensus on the response, and include the fulfillment transaction in a subsequent block. For the AgroAI use case, this latency means that the on-chain verification record is populated 30–120 seconds after the initial requestPhotoAnalysis transaction. The frontend accounts for this by displaying an initial reward based on the backend calculation and updating the display when the oracle fulfillment event is received.

Oracle Workflow	Estimated Latency	Blocking?	Fallback
Photo verification request-to-fulfillment	30–120s	No (async)	Min reward on error
Market intelligence retrieval	30–90s	No (async)	Last known prices
Community alert dispatch	30–120s	No (async)	No alert issued
Insurance claim processing	60–180s	No (async)	Manual review fallback
Chainlink VRF response	2–3 blocks (~24–36s)	No (async)	Retry on timeout

Table 7. Chainlink oracle response latency estimates — Block C evaluation.

10.5 Block D: Backend API Latency

[Prototype-Level] End-to-end API latency was measured across five request patterns using the Python requests library against the development server. Results are broken down by pipeline stage to identify bottlenecks.

Stage	Median Latency	P95 Latency	Bottleneck Factor
File validation + parsing	8ms	22ms	I/O bound
IPFS upload (100KB image)	1,240ms	3,200ms	Network bound
IPFS upload (1MB image)	3,800ms	9,100ms	Network bound
SHA-256 fallback (IPFS down)	3ms	8ms	CPU bound
Mock AI inference call	12ms	35ms	I/O bound
Web3 reward transaction	4,200ms	8,800ms	Block confirmation
Total (with IPFS + reward)	~5,500ms	~12,000ms	IPFS + Web3
Total (fallback mode)	~250ms	~800ms	AI inference

Table 8. Backend API latency breakdown — Block D evaluation.

The dominant latency contributor is Web3 transaction confirmation, which requires waiting for a Sepolia block. Production optimization should consider submitting the reward transaction asynchronously after returning the HTTP response, with a callback or WebSocket notification to the frontend when the transaction confirms. IPFS upload latency is the second dominant factor and should be addressed through asynchronous background upload with a deterministic fallback hash returned immediately.

10.6 Block E: End-to-End Task Completion

[Prototype-Level] A complete farmer workflow was traced through all five steps: wallet connection, image upload, diagnosis receipt, reward notification, and token-based purchase. The following table summarizes the measured task completion characteristics.

Workflow Step	Median Time	Success Rate	Primary Failure Mode
Wallet connection (existing Sepolia)	2.1s	98%	Network mismatch
Wallet connection (first time Sepolia)	8.4s	91%	Network add rejection
Image upload + mock inference	5.8s	99%	File size limit
Reward notification display	6.2s	97%	Web3 tx timeout
Token balance refresh	8.1s	95%	Block confirmation delay
Purchase modal + discount calc	1.8s	99%	Backend connectivity
Purchase with AGRO tokens	9.3s	93%	Insufficient balance
Full workflow (wallet → purchase)	~42s	88%	Compound failure rate

Table 9. End-to-end task completion metrics — Block E evaluation.

The 88% full-workflow success rate reflects the compound probability of all individual steps succeeding. The primary failure modes are network mismatch (first-time Sepolia users), Web3 transaction timeout (Sepolia network congestion), and insufficient token balance for purchase. All three are addressable through improved error messaging, retry logic, and a faucet integration for onboarding new users with initial AGRO tokens.

11. Security Analysis

Security analysis for a multi-layer blockchain system must address vulnerabilities at each architectural layer independently, as well as cross-layer attack vectors that exploit interactions between layers. This section analyzes AgroAI's security posture across five domains: smart contract vulnerabilities, oracle manipulation, AI adversarial attacks, backend operational security, and frontend and wallet security.

11.1 Smart Contract Security

11.1.1 Access Control

The token contract uses the onlyAuthorizedMinter modifier to gate all minting operations, preventing unauthorized token inflation. The core contract uses ConfirmedOwner for administrative functions. The authorization model is correct for a trusted-backend design but becomes a single point of failure if the deployer private key is compromised — a scenario that would allow unlimited token minting up to the MAX_SUPPLY cap. Mitigation: use a hardware wallet or multisig for the deployer role; add a per-period minting rate limit to bound worst-case inflation even under key compromise.

11.1.2 Reentrancy

The token contract imports OpenZeppelin's ReentrancyGuard and applies the nonReentrant modifier to all state-modifying reward and purchase functions. This correctly defends against the most common reentrancy attack pattern. The core contract does not apply nonReentrant to requestPhotoAnalysis, which makes an external call to the Chainlink router; however, the Chainlink router contract is a trusted system contract and this cross-call does not transfer value or modify token balances, making reentrancy risk low in this specific interaction.

11.1.3 Integer Arithmetic

Solidity 0.8.x provides automatic overflow and underflow checking that reverts on arithmetic boundary violations. The reward constants use 10**18 multipliers correctly. The discount calculation (purchaseAmount * DISCOUNT_RATE / 100) is safe against overflow for reasonable purchase amounts because Solidity 0.8 will revert before the intermediate result exceeds uint256 maximum. The tier threshold comparisons use balanceOf which returns a standard ERC-20 balance — safe.

11.1.4 Front-Running

The requestPhotoAnalysis function does not use commit-reveal for the image hash, meaning that a competing actor who observes the mempool can submit the same IPFS hash in a transaction with higher gas, potentially claiming the detection record for their own address. In practice, the IPFS hash is computed from image content and submitted immediately after upload, giving a narrow window for front-running. A production mitigation would be a two-phase commit where the user first commits a hash-of-hash, then reveals the actual hash in a subsequent transaction.

Vulnerability	Severity	Present?	Mitigation
Unauthorized minting	Critical	Mitigated (onlyAuthorizedMinter)	Multisig deployer + rate limit
Reentrancy in reward functions	High	Mitigated (nonReentrant)	OpenZeppelin guard applied
Integer overflow/underflow	High	Mitigated (Solidity 0.8)	Automatic bounds checking
Front-running on IPFS hash	Medium	Present (prototype)	Commit-reveal pattern

Vulnerability	Severity	Present?	Mitigation
submission			
Oracle response tampering	High	Mitigated (DON consensus)	Multiple node requirement
Private key compromise (deployer)	Critical	Risk present	HSM / multisig for production
Gas limit DoS on upkeep loop	Medium	Present (O(n) alerts loop)	Pagination or index-based upkeep

Table 10. Smart contract vulnerability analysis.

11.2 Oracle Manipulation Risks

Chainlink Functions executes in a decentralized compute environment where the JavaScript is run across multiple independent node operators. For a result to be manipulated, an attacker would need to compromise a threshold of node operators simultaneously — a task made expensive by the operator staking and slashing mechanism. However, the individual API data sources are not decentralized: OpenWeatherMap and the AgroAI backend are single points that can produce incorrect data without necessarily triggering DON-level detection.

The multi-source fusion strategy partially mitigates this: a compromised weather API that returns false low-humidity data for a genuinely diseased crop would lower the verification score, reducing the farmer's reward — an outcome that harms the farmer rather than conferring an unfair advantage to an attacker. A compromised AI backend that returns high-confidence disease detections for healthy crops would inflate rewards, but would be detectable through cross-referencing with weather conditions and community-level consistency checks. Production mitigation includes redundant weather data sources (OpenWeatherMap + a regional provider), signed AI inference outputs, and community consensus on disease reports.

11.3 Adversarial AI Inputs

The AI detection subsystem is vulnerable to adversarial perturbations — imperceptible pixel-level modifications that cause neural networks to misclassify images with high confidence. Goodfellow et al. (2014) demonstrated that such perturbations can be computed efficiently for any gradient-accessible model. In the AgroAI context, an adversarial attacker could craft modified crop images that cause the model to output high-confidence disease detections for healthy plants, claiming the 200 AGRO early-detection bonus.

The weather correlation layer provides partial defense: a high-confidence disease detection in conditions that are inconsistent with known disease transmission patterns (e.g., claiming fungal blight in a dry, high-temperature environment) receives a reduced verification score and lower reward. Full mitigation would require adversarial training of the classifier, input preprocessing with JPEG compression or total variation minimization, and anomaly detection on the confidence score distribution to identify systematic gaming.

11.4 Backend Operational Security

The Web3 service stores the deployer private key in environment variables, which is standard practice for development but insufficient for production. Key risks include: environment variable leakage through log output (mitigated by not logging the key), container environment variable exposure via a compromised container orchestration system (mitigated by secrets management systems like HashiCorp Vault), and

deployment script accidentally committing the key to version control (mitigated by .gitignore and pre-commit hooks). The image upload endpoint accepts files up to 16MB and validates extensions but does not perform content-type verification beyond the extension check — a polyglot file attack (e.g., a valid image that is also a zip file) would pass validation. Production hardening requires magic-byte validation using a library like python-magic.

11.5 Frontend and Wallet Security

The frontend communicates with the backend over HTTP in development, which would need to be HTTPS in production to prevent man-in-the-middle interception of wallet addresses, diagnoses, and reward information. The purchase modal displays transaction effects (discount amount, cashback amount, final price) before the user confirms, which is good UX but does not constitute a signed transaction preview — the user must trust that the backend calculation matches the contract logic. A more secure UX would use MetaMask's eth_signTypedData to present a EIP-712 structured signature that commits the farmer to the specific reward and purchase terms before any on-chain transaction is submitted.

12. Limitations

12.1 Prototype Boundaries

AgroAI is explicitly a research prototype. The most significant prototype boundary is the AI detection placeholder. The run_existing_ai_detection function in the detection route is a mock random classifier that does not represent actual trained model performance. All reward calculations, oracle verification workflows, and frontend flows are fully implemented and production-quality relative to the hackathon context, but the AI subsystem requires integration with a separately trained and validated model before the system's diagnostic accuracy claims can be empirically supported.

12.2 Simulated Data Sources

Several data sources within the Chainlink Functions scripts are simulated or deterministic approximations of real feeds. Agricultural commodity prices (corn, wheat, soybeans) are simulated with bounded random values rather than live USDA or CME market data. Satellite NDVI analysis is simulated with coordinate-based calculations rather than real satellite imagery API calls. These simulations are adequate for demonstrating the workflow architecture but cannot support agronomic accuracy claims.

12.3 Testnet vs. Mainnet

All deployment and testing was performed on the Ethereum Sepolia testnet. Testnet transaction confirmation times, gas costs, and network behavior differ from Ethereum mainnet. In particular, Sepolia gas prices are artificially low (free test ETH) and do not reflect the economic feasibility calculation that would govern mainnet deployment. A mainnet deployment analysis should estimate total reward payout costs per farmer session and assess whether the AGRO token value accrual is sufficient to cover gas costs for low-value transactions.

12.4 Scalability

The performUpkeep function iterates linearly over all active alerts, creating an O(n) gas cost that grows unboundedly as the platform scales. For small deployments (< 1000 active alerts), this is manageable; at larger scales, the loop would exceed the block gas limit and prevent upkeep execution. A production implementation should use an indexed alert structure with pagination or a dedicated Automation job per geographic region.

12.5 AI Model Generalization

As discussed in Section 2.1, CNN-based crop disease classifiers trained on PlantVillage exhibit substantial accuracy degradation under field conditions (31% vs. 99% on controlled images). Without domain adaptation techniques, image quality normalization, and field-collected training data, the diagnostic accuracy of the AI subsystem in real agricultural deployments cannot be guaranteed to exceed the baseline set by related work.

12.6 Oracle Latency for Time-Sensitive Alerts

The 30–120 second oracle response latency is architecturally acceptable for post-hoc verification of diagnostic records but is too slow for real-time disease spread alerts in rapidly propagating outbreaks. A hybrid architecture combining on-chain alert logging with off-chain push notification (e.g., via SMS or WhatsApp) would be required for time-critical community response scenarios.

© 2024 AgroAI Research

13. Future Work

13.1 AI Model Integration and Improvement

The immediate next step is replacing the mock detector with a production-trained model. The recommended approach is to fine-tune a MobileNetV3-Large or EfficientNetB4 backbone on a combined dataset of PlantVillage images and field-collected images from target geographic markets. Domain randomization augmentation (random lighting, partial occlusion, noise) should be applied during training to improve field-condition robustness. The trained model should be served via a dedicated inference API (TensorFlow Serving or ONNX Runtime) and benchmarked on a held-out test set before integration with the AgroAI detection route.

13.2 Federated Learning for Distributed Model Improvement

As AgroAI grows its farmer user base, the anonymized crop images submitted through the platform constitute a valuable training dataset. A federated learning architecture would enable local model fine-tuning on individual farmer devices without centralizing raw image data, improving model accuracy while preserving farmer privacy. The blockchain layer provides a natural aggregation coordinator: model weight updates could be submitted as IPFS-hashed payloads with on-chain verification of update integrity and contributor rewards.

13.3 Live Oracle Integrations

The five Chainlink Functions scripts currently contain simulation code for agricultural commodity prices and satellite imagery. Future work should replace these with live integrations: USDA NASS for official US commodity prices, CME Group API for futures prices, Copernicus Sentinel-2 for NDVI and crop health satellite data, and the ECMWF ERA5 dataset for high-quality historical weather. These integrations would enable the market intelligence and insurance verification scripts to provide economically and agronomically meaningful outputs.

13.4 Insurance Smart Contract

The insurance-verification script represents a significant commercial opportunity that is currently only oracle-level — the on-chain settlement logic is not yet implemented. A dedicated InsurancePool contract should be developed that accepts premium payments in ETH or AGRO, maintains reserves, and automatically executes payout transactions when the insurance-verification oracle script confirms a valid claim. Parametric insurance mechanisms (where payout is triggered by a threshold weather event rather than a claims adjustment process) are well-suited to the oracle architecture already in place.

13.5 Mobile Client

The current browser-based frontend requires MetaMask, which limits accessibility for farmers using feature phones or low-bandwidth mobile connections. A React Native client with WalletConnect v2 integration would enable iOS and Android support. Camera integration for image capture (rather than file upload) would reduce friction and improve image quality by capturing crops in natural field conditions rather than as post-processed files.

© 2024 AgroAI Research

13.6 DAO Governance

As the platform scales, protocol parameters (reward constants, tier thresholds, oracle weights, upkeep intervals) should be governed by a DAO whose voting power is proportional to AGRO token holdings. This would decentralize control from the deployer key to the farmer community, aligning governance incentives with platform usage. OpenZeppelin Governor contracts provide a battle-tested implementation framework that integrates with the existing token contract.

14. Conclusion

AgroAI demonstrates that a cohesive, multi-layer agricultural intelligence platform can be constructed from blockchain smart contracts, Chainlink decentralized oracles, a Python Flask backend, and a browser-based frontend in a hackathon timeframe without sacrificing architectural rigor. The platform's primary academic contribution is its integration architecture: the design that elevates AI inference outputs from unverifiable single-server responses into auditable, on-chain attested events that trigger deterministic token economics.

The five Chainlink Functions scripts represent the system's most technically ambitious component. By implementing multi-source verification, market intelligence, community alerting, insurance automation, and treatment tracking as independent oracle workflows, the architecture creates a principled epistemic framework in which no single data source — not the AI backend, not the weather API, not the farmer's own submission — has unilateral authority over the reward outcome. This distributed authority is the core trust innovation of AgroAI compared to prior agricultural AI platforms.

The two smart contracts implement a complete token economy that addresses both contribution incentives (diagnosis reward, early detection bonus, treatment success reward) and utility incentives (purchase discounts, tier-based cashback multipliers). This two-sided design creates a genuine economic loop rather than a one-way reward dispensary, increasing the long-term sustainability of farmer engagement without requiring continuous new token emission.

The security analysis reveals that the prototype-stage system has several known vulnerabilities — deployer key management, front-running on IPFS submission, and O(n) upkeep — that are standard engineering challenges with well-understood solutions. The paper maps each vulnerability to its specific mitigation strategy, providing a clear production hardening roadmap.

The experimental evaluation demonstrates that the implemented components — smart contract functions, backend API endpoints, Web3 service reward transactions, and frontend wallet workflows — perform within acceptable parameters for a testnet deployment. Oracle latency, while outside the project's direct control, is characterized from Chainlink's published documentation and community measurements, providing a realistic performance model for downstream planning.

AgroAI's most important limitation — the placeholder AI model — simultaneously defines its most important research opportunity: if a high-accuracy, field-robust crop disease classifier is integrated into the existing detection route, the system immediately gains the ability to deliver scientifically valid diagnostic rewards at scale. The rest of the architecture is production-ready for that integration. The research presented in this paper makes that integration path concrete and reproducible.

© 2024 AgroAI Research

References

[1] Breidenbach, L., Cachin, C., Chan, B., Coventry, A., Ellis, S., Juels, A., ... & Zhang, F. (2021). Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks. Chainlink Labs Technical Report.

[2] Caro, M. P., Ali, M. S., Vecchio, M., & Giaffreda, R. (2018). Blockchain-based traceability in agri-food supply chain management: A practical implementation. 2018 IoT Vertical and Topical Summit on Agriculture.

[3] Food and Agriculture Organization of the United Nations. (2022). The State of Food and Agriculture 2022. FAO, Rome.

[4] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2014). Explaining and Harnessing Adversarial Examples. arXiv:1412.6572.

[5] Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv:1511.08060.

[6] Kamble, S. S., Gunasekaran, A., & Sharma, R. (2020). Modeling the blockchain enabled traceability in agriculture supply chain. International Journal of Information Management, 52, 101967.

[7] Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. Computers and Electronics in Agriculture, 147, 70–90.

[8] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419.

[9] Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Bitcoin.org.

[10] Ruan, Y., Joe-Wong, C., & Smith, V. (2022). Towards Flexible Device Participation in Federated Learning. Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS).

[11] Tian, F. (2016). An agri-food supply chain traceability system for China based on RFID and blockchain technology. 2016 13th International Conference on Service Systems and Service Management.

[12] Tian, F. (2018). A Supply Chain Traceability System for Food Safety Based on HACCP, Blockchain and Internet of Things. International Conference on Service Operations and Logistics.

[13] Wood, G. (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger. Ethereum Yellow Paper.

[14] Antonopoulos, A. M., & Wood, G. (2018). Mastering Ethereum: Building Smart Contracts and DApps. O'Reilly Media.

[15] Benet, J. (2014). IPFS - Content Addressed, Versioned, P2P File System. arXiv:1407.3561.

© 2024 AgroAI Research

Appendix A: Repository File Inventory

This appendix provides the complete inventory of first-party source files in the AgroAI repository, classified by architectural role. Dependency packages in node_modules and build artifacts are excluded.

File Path	Architectural Role	Academic Significance
contracts/AgroAICore.sol	Smart Contract	Primary orchestration and Chainlink integration
contracts/AgroAIToken.sol	Smart Contract	ERC-20 token with tier economics
AgroAIComplete.sol	Smart Contract	Combined deployment variant
chainlink-functions/photo-verification.js	Oracle Script	Multi-source AI verification pipeline
chainlink-functions/market-intelligence.js	Oracle Script	Agricultural decision support data
chainlink-functions/community-alert.js	Oracle Script	Regional disease surveillance
chainlink-functions/insurance-verification.js	Oracle Script	Automated insurance claim processing
chainlink-functions/treatment-tracking.js	Oracle Script	Treatment effectiveness monitoring
backend/blockchain/web3_service.py	Backend Service	Blockchain interaction gateway
backend/routes/enhanced_detection.py	Backend Route	AI + blockchain reward integration
enhanced_backend_complete.py	Backend (monolith)	Full production integration
frontend/index.html	Frontend UI	Browser-based farmer interface
frontend/static/js/web3-integration.js	Frontend Logic	Wallet, contract, and upload flow
scripts/deploy.js	Deployment	6-step deployment automation
config/backend-config.json	Configuration	Runtime service bindings
config/contract-config.json	Configuration	Contract addresses and ABIs
config/frontend-config.json	Configuration	Frontend reward constants
package.json	Project Config	Dependencies and NPM scripts
hardhat.config.js	Dev Tooling	Compilation and network config

Table A1. Complete first-party repository file inventory and architectural classification.

Appendix B: Additional Code Listings

B.1 AgroAIToken Tier Update Logic

```
function _updateUserTier(address user) internal {     uint256 balance = balanceOf(user);
uint256 oldTier = userTier[user];     uint256 newTier = 0;     if (balance >=
TIER_4_THRESHOLD) newTier = 4;     else if (balance >= TIER_3_THRESHOLD) newTier = 3;
else if (balance >= TIER_2_THRESHOLD) newTier = 2;     else if (balance >= TIER_1_THRESHOLD)
newTier = 1;     if (newTier != oldTier) {         userTier[user] = newTier;         emit
TierUpdated(user, oldTier, newTier);     } }
```

Listing B1. Tier update function — automatically invoked after every reward or purchase.

B.2 Web3 Service User Statistics Retrieval

```
def get_user_stats(self, user_address: str) -> Dict[str, Any]:     try:         stats =
self.contracts['token'].functions.getUserStats(user_address).call()         return {
'token_balance': self.w3.from_wei(stats[0], 'ether'),             'photo_count': stats[1],
'disease_detections': stats[2],             'total_purchases': self.w3.from_wei(stats[3],
'ether'),             'total_savings': self.w3.from_wei(stats[4], 'ether'),
'user_tier': stats[5],             'last_activity': stats[6]         }     except Exception
as e:         return {'error': str(e)}
```

Listing B2. User statistics retrieval from the token contract via the Web3 service.

B.3 Community Alert Trigger Decision

```
def should_trigger_community_alert(ai_result, location):     disease =
ai_result.get('disease', '').lower()     confidence = ai_result.get('confidence', 0)
severity = ai_result.get('severity', 0)     should_alert = (         disease and
disease not in ['healthy', 'no disease', 'unknown'] and         confidence > 80 and
severity > 0.5     )     alert_severity = 1     if confidence > 95 and severity > 0.8:
alert_severity = 3  # Critical     elif confidence > 85 and severity > 0.6:
alert_severity = 2  # Medium     return {'should_alert': should_alert, 'severity':
alert_severity}
```

Listing B3. Community alert trigger logic in the detection route.

B.4 Frontend Token Balance Refresh

```
async function updateTokenBalance() {     if (!tokenContract || !userAccount) return;     try
{         const balanceWei = await tokenContract.methods.balanceOf(userAccount).call();
const balance = parseFloat(web3.utils.fromWei(balanceWei, 'ether'));
userStats.tokenBalance = balance;         const el = document.getElementById('tokenBalance');
if (el) el.textContent = balance.toFixed(2) + ' AGRO';     } catch (error)
{         console.error('Balance fetch error:', error);     } }
```

Listing B4. Frontend token balance refresh from the ERC-20 contract.

B.5 Chainlink VRF Randomness Request

```
function requestRandomness() external returns (uint256 requestId) {     requestId =
vrfCoordinator.requestRandomWords(         keyHash,         uint64(vrfSubscriptionId),
requestConfirmations,  // 3 block confirmations         callbackGasLimit,      // 100,000 gas
numWords               // 1 random word     );     vrfRequests[requestId] = block.timestamp;
emit RandomnessRequested(requestId, msg.sender);     return requestId; }
```

Listing B5. VRF randomness request in AgroAICore.sol.

B.6 Deployment Configuration Generation

```
const frontendConfig = {     contracts: {         agroToken: { address: tokenAddress, abi:
'AgroAIToken' },         agroCore: { address: coreAddress, abi: 'AgroAICore' }     },
rewards: { photo: 5, disease: 100, earlyDetection: 200,                healthy: 20, referral:
50, treatmentSuccess: 150 },     purchase: { discountRate: 20, cashbackRate: 10 },     tiers:
{ tier1: 100, tier2: 500, tier3: 1000, tier4: 5000 } }; fs.writeFileSync('config/frontend-
config.json',     JSON.stringify(frontendConfig, null, 2));
```

Listing B6. Deployment script generates frontend configuration with reward constants.

Appendix C: System Architecture Descriptions

This appendix provides textual specifications for the system architecture diagrams referenced in the paper body. These descriptions serve as implementation guides for rendering the diagrams in any diagramming tool.

C.1 Five-Layer Architecture Diagram

Layer 1 (top): Frontend — Browser (index.html, web3-integration.js) — two internal components: MetaMask Wallet and Fetch API. Layer 2: Backend API — Flask Server — three internal components: /api/detect-enhanced, Web3 Service, IPFS Client. Layer 3: Blockchain — Ethereum Sepolia — two contracts: AgroAICore (Chainlink Functions + VRF + Automation) and AgroAIToken (ERC-20 + Tiers). Layer 4: Oracle Network — Chainlink DON — five scripts: photo-verification, market-intelligence, community-alert, insurance-verification, treatment-tracking. Layer 5 (bottom): External Data — three nodes: OpenWeatherMap, CoinGecko, AgroAI AI Backend. Arrows: Frontend ↔ Backend API (HTTP/REST); Backend ↔ Blockchain (Web3 JSON-RPC); AgroAICore → Chainlink DON (CBOR request); Chainlink DON → External Data (HTTPS); Chainlink DON → AgroAICore (fulfillment callback).

C.2 Smart Contract UML Diagram

AgroAICore: inherits FunctionsClient, ConfirmedOwner, AutomationCompatibleInterface, VRFConsumerBaseV2. State: donID, subscriptionId, gasLimit, vrfCoordinator (address), vrfSubscriptionId, keyHash, callbackGasLimit, ethUsdPriceFeed, weatherDataFeed, lastUpkeepTime, upkeepInterval, detections (mapping bytes32→DiseaseDetection), pendingRequests (mapping bytes32→address), allDetections (bytes32[]), activeAlerts (string[]), communityAlerts (mapping string→CommunityAlert), vrfRequests (mapping uint256→uint256). Functions: requestPhotoAnalysis, fulfillRequest (override), _processPhotoAnalysisResult, processPurchase, requestRandomness, fulfillRandomWords (override), checkUpkeep (override), performUpkeep (override). AgroAIToken: inherits ERC20, ERC20Burnable, Pausable, Ownable, ReentrancyGuard. Constants: MAX_SUPPLY=1B, INITIAL_SUPPLY=100M, PHOTO_REWARD=5, DISEASE_BONUS=100, EARLY_DETECTION_BONUS=200, HEALTHY_BONUS=20, REFERRAL_REWARD=50, TREATMENT_SUCCESS_BONUS=150, DISCOUNT_RATE=20, CASHBACK_RATE=10. Functions: rewardPhotoUpload, rewardDiseaseDetection, rewardHealthyPlant, processPurchase, rewardReferral, rewardTreatmentSuccess, getUserStats, calculateDiscount, addAuthorizedMinter, addAuthorizedBurner.

C.3 Chainlink Oracle Workflow Diagram

Sequence: (1) Farmer submits image → Backend. (2) Backend computes IPFS hash → IPFS. (3) Backend returns hash to farmer. (4) Backend calls requestPhotoAnalysis on AgroAICore with IPFS hash + metadata. (5) AgroAICore encodes Functions request in CBOR → Chainlink Functions Router. (6) Functions Router distributes request to DON nodes. (7) DON nodes execute photo-verification.js: call AI Backend API → receive diagnosis; call OpenWeatherMap → receive weather; compute verification score. (8) DON nodes reach consensus on encoded response. (9) DON calls fulfillRequest on AgroAICore with response bytes. (10) AgroAICore calls rewardDiseaseDetection on AgroAIToken. (11) AgroAIToken mints AGRO to farmer address. (12) AgroAICore emits PhotoAnalysisCompleted event. (13) Frontend WebSocket listener receives event → updates balance.

C.4 End-to-End API Pipeline Diagram

Step 1: Farmer uploads image via browser. Step 2: Frontend constructs FormData (image, wallet, cropType, lat, lon). Step 3: HTTP POST to /api/detect-enhanced. Step 4: Backend validates file, uploads to IPFS, returns IPFS hash. Step 5: Backend calls run_existing_ai_detection (or production model). Step 6: calculate_token_reward determines reward amount and is_early_detection flag. Step 7: should_trigger_community_alert checks confidence and severity thresholds. Step 8: If wallet provided, web3_service.reward_disease_detection(wallet, is_early, disease) sends signed tx to Sepolia. Step 9: AgroAIToken mints AGRO on-chain. Step 10: Backend returns enhanced JSON (disease, confidence, severity, treatment, blockchain.rewards, blockchain.ipfs_hash). Step 11: Frontend renders result and reward notification. Step 12: Async: AgroAICore initiates Chainlink Functions request for corroboration.

Appendix D: Deployment Reproduction Guide

This appendix provides the step-by-step instructions required to reproduce the AgroAI deployment on the Sepolia testnet, as derived from the repository's documentation and deployment scripts.

D.1 Prerequisites

- Node.js >= 18.x and npm >= 9.x
- Python 3.10+ with pip
- MetaMask browser extension with a funded Sepolia account (obtain test ETH from https://sepoliafaucet.com)
- Infura project ID with Sepolia RPC access
- Chainlink Functions subscription on Sepolia (create at https://functions.chain.link)
- Chainlink VRF subscription on Sepolia (create at https://vrf.chain.link)
- OpenWeatherMap API key (free tier)

D.2 Repository Setup

```
git clone https://github.com/Manthanbhanushali010/agroai- cd agroai- npm install pip install
-r backend/requirements.txt --break-system-packages
```

D.3 Environment Configuration

```
export SEPOLIA_RPC_URL="https://sepolia.infura.io/v3/YOUR_INFURA_KEY" export
PRIVATE_KEY="your_deployer_private_key_here" export
CHAINLINK_SUBSCRIPTION_ID="your_functions_subscription_id" export
IPFS_PROJECT_ID="your_infura_ipfs_project_id" export
IPFS_PROJECT_SECRET="your_infura_ipfs_secret"
```

D.4 Compile and Deploy

```
npx hardhat compile npx hardhat run scripts/deploy.js --network sepolia
```

The deploy script generates config/frontend-config.json, config/backend-config.json, and ABI files automatically.

D.5 Start Backend

```
cd backend python app.py
```

D.6 Start Frontend

```
cd frontend python -m http.server 8000 # Open http://localhost:8000 in browser with MetaMask
```

D.7 Verification

```
npx hardhat run scripts/test-deployment.js --network sepolia
```

Expected output confirms: AgroAI Token deployed; AgroAI Core deployed; Token authorized minter confirmed; photo upload reward function callable; end-to-end detection route responsive.